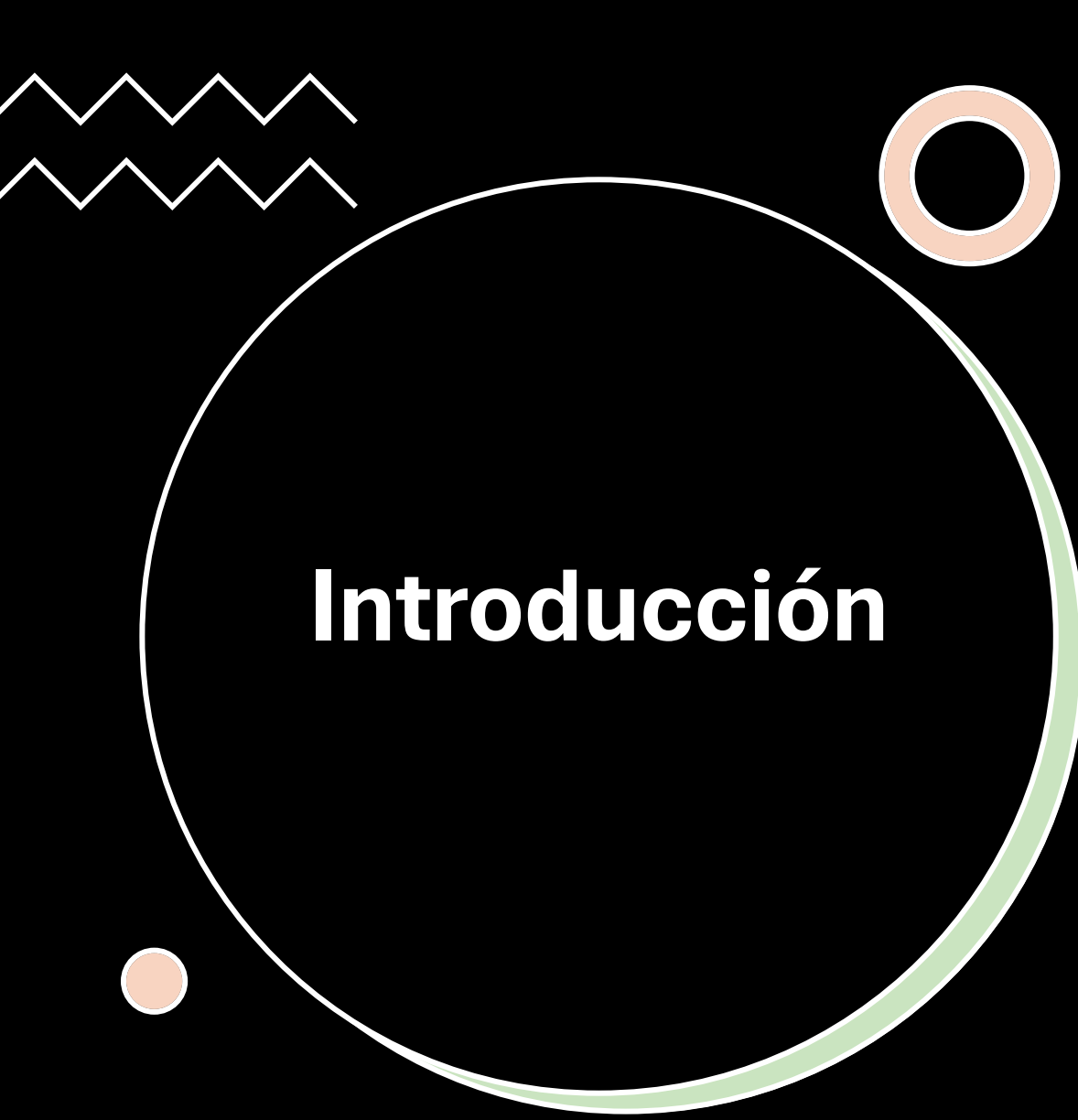




Red Neuronal para la Clasificación de la Flor Iris

Angel Vazquez



Introducción

¿Qué es la flor Iris?

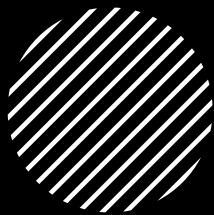
El conjunto de datos de la flor Iris es un conjunto clásico en Machine Learning que contiene mediciones de 150 flores de tres especies:

- Iris Setosa
- Iris Versicolor
- Iris Virginica





Cada flor tiene
cuatro
características:



- Largo del sépalo
- Ancho del sépalo
- Largo del pétalo
- Ancho del pétalo
- El objetivo es clasificar las flores en una de las tres especies usando una red neuronal.

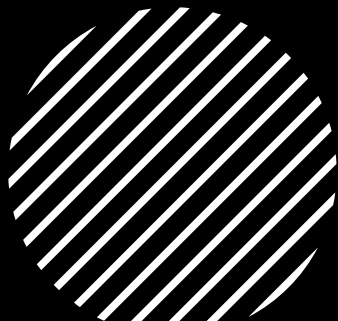




Estructura de la Red Neuronal

La red neuronal utilizada tiene:

- **4 neuronas de entrada** (una por cada característica de la flor)
- **10 neuronas en la capa oculta**
- **3 neuronas en la capa de salida** (una por cada especie de Iris)





Representación de las Salidas

Para representar las tres especies de flores, usamos una codificación binaria:

- **Iris Setosa:** [1, 0, 0]
- **Iris Versicolor:** [0, 1, 0]
- **Iris Virginica:** [0, 0, 1]
- Esto permite que la red neuronal aprenda a clasificar correctamente cada observación.



Inicialización de la Red

- Antes de entrenar, se inicializan los pesos y sesgos con valores aleatorios:
- `W1 = rand(n_entradas, n_ocultas); // Pesos capa oculta`
- `b1 = rand(1, n_ocultas); // Sesgos capa oculta`
- `W2 = rand(n_ocultas, n_salidas); // Pesos capa salida`
- `b2 = rand(1, n_salidas); // Sesgos capa salida`
- Esto permite que la red comience con una distribución de valores que se irán ajustando durante el entrenamiento.



Algoritmo de Entrenamiento

El entrenamiento se realiza mediante **retropropagación del error**:

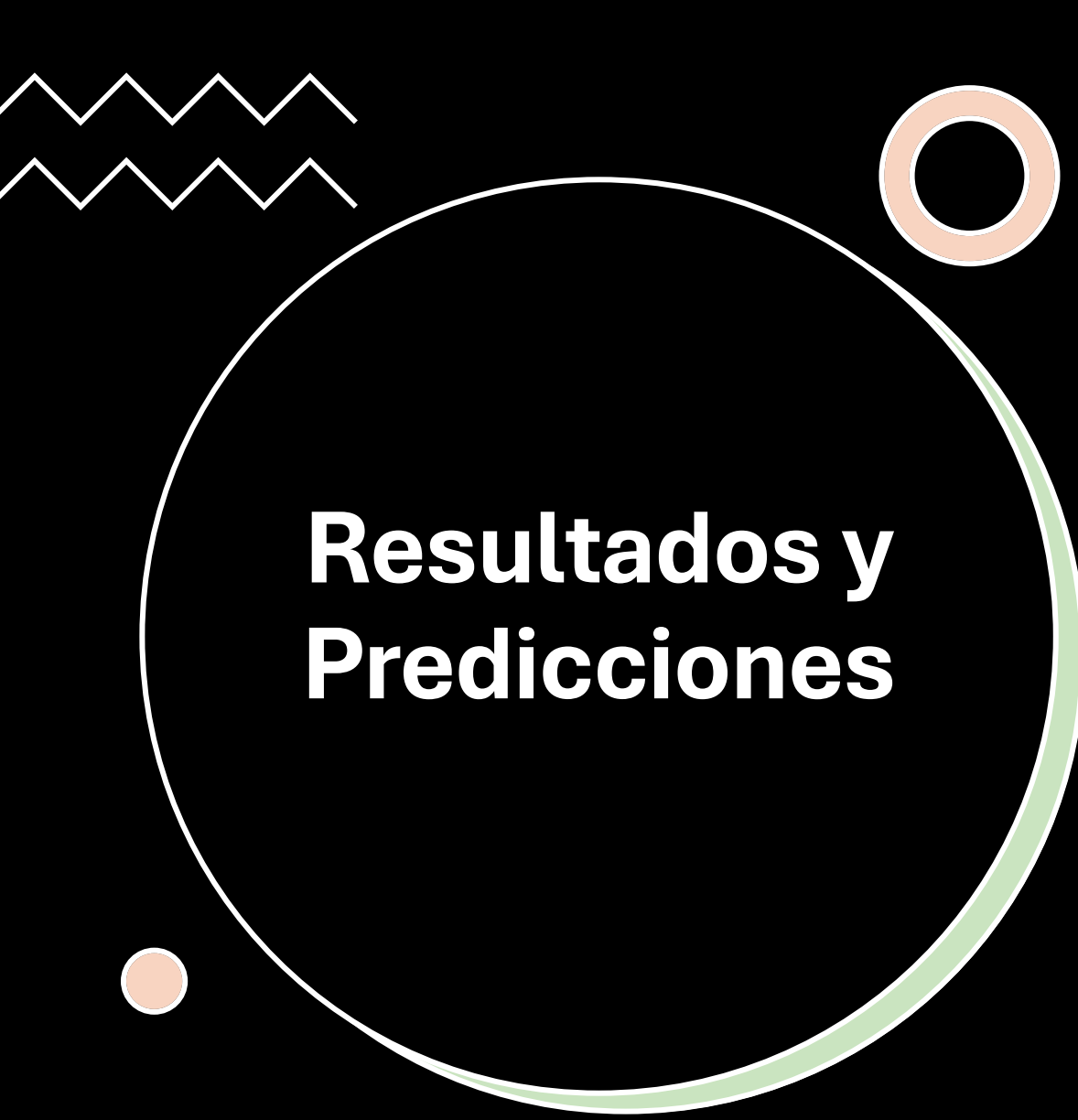
- 1. Propagación hacia adelante:** Se calculan las salidas de cada capa.
 - 2. Cálculo del error:** Se compara la salida esperada con la obtenida.
 - 3. Retropropagación:** Se ajustan los pesos para minimizar el error.
 - 4. Actualización de pesos:** Se aplica la regla de aprendizaje para mejorar la red.
- Este proceso se repite durante múltiples iteraciones hasta que el error sea mínimo.





Implementación del Entrenamiento

- El entrenamiento de la red se realiza con un bucle donde se actualizan los pesos en cada iteración:
 - ```
for iter = 1:max_iter
```
  - ```
    Z1 = X * W1 + b1_expanded;
```
 - ```
 A1 = sigmoid(Z1);
```
  - ```
    Z2 = A1 * W2 + b2_expanded;
```
 - ```
 A2 = sigmoid(Z2);
```
  - ```
    error = Y - A2;
```
 - ```
 // Retropropagación
```
  - ```
    dZ2 = error .* sigmoid_derivada(Z2);
```
 - ```
 dW2 = A1' * dZ2;
```
  - ```
    db2 = sum(dZ2, 1);
```
 - ```
 dZ1 = (dZ2 * W2') .* sigmoid_derivada(Z1);
```
  - ```
    dW1 = X' * dZ1;
```
 - ```
 db1 = sum(dZ1, 1);
```
  - ```
    // Actualización de pesos
```
 - ```
 W2 = W2 + tasa_aprendizaje * dW2;
```
  - ```
    b2 = b2 + tasa_aprendizaje * db2;
```
 - ```
 W1 = W1 + tasa_aprendizaje * dW1;
```
  - ```
    b1 = b1 + tasa_aprendizaje * db1;
```
 - ```
end
```
- 



# Resultados y Predicciones

- Una vez entrenada, la red se prueba con los mismos datos para evaluar su rendimiento:
- `Y_pred = sigmoid(sigmoid(X * W1 + b1_exp) * W2 + b2_exp);`
- `disp("Predicciones:");`
- `disp(cat(2,X,fix(Y_pred+0.5)));`
- Las predicciones se comparan con las etiquetas originales para verificar la precisión del modelo.

